Ruprecht-Karls-Universität Heidelberg
Institut für Informatik

Bachelor Thesis

Parallel ASP Planning

Name: Levi Maximilian Klein
Student ID: 4283957
Supervisor: Dr. Gnad
Submission date: 17.02.2026

Ich versichere, dass ich diese Bachelor-Arbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Heidelberg, dd.mm.yyyy

Zusammenfassung

Dies ist eine Zusammenfassung der Arbeit.

Abstract

This is the abstract.

Contents

List of Figures

List of Tables

1 Introduction

This chapter contains an overview of the topic as well as the goals and contributions of your work.

1.1 Motivation

1.2 Goal of the Thesis

1.3 Structure of the Thesis

Here you describe the structure of the thesis. For example:
In Kapitel 2 werden grundlegende Methoden für diese Arbeit vorgestellt.

2 Theory

2.1 Definition

Definition 1 ***Multi-valued planning task.*** A STRIPS-like multivalued planning task according to (Helmert 2006) is given by a 4 tuple $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$, where:

- $\mathcal{F}$ is a finite set of state variables, also called **fluents**. Each $x \in \mathcal{F}$ has a finite domain and in every state, x takes exactly one value from its domain.
- s_0, the **initial state**, is a total function,, s.t. $s_0(x) \in dom(x)$ for each $x \in \mathcal{F}$, i.e. a function that maps each fluent to a value from its domain.
- s_*, the **goal conditions**, is a partial function, s.t. $s_*(x) \in dom(x)$, for each $x \in dom(s_*)$ i.e. a function that specifies the desired values for a subset of fluents.
- $\mathcal{O}$ is a finite set of operators, also called **actions**. We define an action a as $\langle a^c, a^e \rangle$, where a^c is the precondition and a^e is the postcondition of a. Both a^c and a^e are partial functions, s.t. $a^c(x) \in dom(x)$ for each $x \in dom(a^c)$ and $a^e(x) \in dom(x)$ for each $x \in dom(a^e)$.

Remark. An action a also has a finite set of effects eff, where $eff = \langle cond, x, v \rangle$. $cond$, the effect condition, is a (possibly empty) partial function, s.t. $cond(x) \in dom(x)$ for each $x \in dom(cond)$. $x \in \mathcal{F}$ is the effected fluent and $v \in dom(x)$ is the value assigned to x if $cond$ holds.

Example 1. Let us illustrate the definitions with a small example. The plan is to leave the house by opening the door, going outside, and finally closing the door.

- $\mathcal{F} = \{door, location\}$ with $dom(door)$=$\{open, closed\}$, $dom(location)$=$\{inside, outside\}$
- Initial state: $s_0 = \{door{=}closed, location{=}inside\}$
- Goal: $s_* = \{door{=}closed, location{=}outside\}$
- Actions: $\mathcal{O} = \{openDoor, closeDoor, goOut\}$, where

$$openDoor = \langle \{door{=}closed\}, \{door{=}open\} \rangle$$
$$closeDoor = \langle \{door{=}open\}, \{door{=}closed\} \rangle$$
$$goOut = \langle \{location{=}inside, door{=}open\}, \{location{=}outside\} \rangle$$

Definition 2 ***Successor state.*** Let s be state and $a \in \mathcal{O}$ an action.
The *successor state* $o(a, s)$, obtained by applying action a in state s, is defined if the precondtion

of a holds, i.e., $a^c(x) = s(x)$ for all $x \in dom(a^c)$. Then

$$s'(x) = o(a, s) = \begin{cases} a^e(x) & \text{if } x \in dom(a^e) \\ s(x) & \text{otherwise.} \end{cases}$$

Remark. Let $A_n = \langle a_1, \ldots, a_n \rangle$ be a sequence of actions with length n. We define the application of A_n to a state s recursevily by

$$o(A_n, s) = o(a_n, o(A_{n-1}, s))$$

if $o(A_i, s)$ is defined for all $1 \leq i \leq n$.

Definition 3 ***Sequential plan.*** A *sequential plan* is a sequence of actions $A_n = \langle a_1, \ldots, a_n \rangle$, s.t. $s' = o(A_n, s_0)$ is defined and $s'(x) = s_*(x)$ for all $x \in dom(s_*)$.

Example 2. Consider the planning task from the previous example. One possible sequential plan is $A = \langle openDoor, goOut, closeDoor \rangle$. Since

$$\begin{aligned} o(A, s_0) &= o(\langle openDoor, goOut, closeDoor \rangle, \{door{=}closed, location{=}inside\}))) \\ &= o(\langle goOut, closeDoor \rangle, \{door{=}open, location{=}inside\})) \\ &= o(closeDoor, \{door{=}open, location{=}outside\}) \\ &= \{door{=}closed, location{=}outside\} = s_* \end{aligned}$$

For parallel actions and plans, we first need the definition of a confluent set of actions.

Definition 4 ***confluent.*** Let $A = \{a_1, \ldots, a_n\} \subseteq \mathcal{O}$ be a set of actions.
A is *confluent* $:\Longleftrightarrow a_i^e(x) = a_j^e(x)$ for all $1 \leq i < j \leq n$ and each $x \in dom(a_i^e) \cap dom(a_j^e)$

Remark. A is confluent $\Leftrightarrow$ B is confluent for all $B \subseteq A$

Example 3. In our example $\{openDoor, goOut\}$ and $\{closeDoor, goOut\}$ are confluent sets.
We extend the example with the fluents *light* and *key* where $dom(light) = dom(key) = \{0, 1\}$ and two actions *lightsOut* and *getKey*:

$$lightsOut = \langle \{location{=}inside, light{=}1\}, \{light{=}0\} \rangle$$
$$getKey = \langle \{location{=}inside, key{=}0\}, \{key{=}1\} \rangle$$

Additionally, we modify the precondition: $openDoor^c = \{door{=}closed, key{=}1\}$ and our new goal is $s_* = \{location{=}outside, door{=}closed, light{=}0\}$.
Now for example $\{getKey, openDoor, goOut, lightsOut\}$ and $\{getKey,\ closeDoor,\ goOut,\ lightsOut\}$ are also confluent sets of actions.

Definition 5 ***Parallel representations.*** Let s be a state and $A = \{a_1, \ldots, a_n\}$ be a confluent set of actions. Then A is

(i) $\forall$*-step serializable* in $s :\Longleftrightarrow$ $o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)}\rangle, s)$ is defined for all permutations π

(ii) $\exists$*-step serializable* in $s :\Longleftrightarrow$ $o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)}\rangle, s)$ is defined for some permutation π and $a^c(x) = s(x)$ for each $a \in A$ and $x \in dom(a^c)$

(iii) *relaxed* $\exists$*-step serializable* in $s :\Longleftrightarrow$ $o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)}\rangle, s)$ is defined for some permutation π

Remark. A is $\forall$-step serializable $\Rightarrow$ A is $\exists$-step serializable $\Rightarrow$ A is relaxed $\exists$-step serializable.

Definition 6 ***Parallel Plans.*** Let $A = \langle A_1, \ldots, A_n\rangle$ be a sequence of confluent sets of actions. A is a $\forall$*-step,* $\exists$*-step or relaxed* $\exists$*-step plan* iff

- $s_n(x) = s_*(x)$ for each $x \in dom(s_*)$
- A_i is $\forall$-step, $\exists$-step or relaxed $\exists$-step serializable in s_{i-1} for $1 \leq i \leq n$, where
  $s_i(x) = \begin{cases} a^e(x) & \text{if } x \in dom(a^e) \\ s_{i-1}(x) & \text{else} \end{cases}$ for each $a \in A_i$

Example 4. In our modified example we have:

- $\forall$-step plan: $\langle\{getKey, lightsOut\}, \{openDoor\}, \{goOut\}, \{closeDoor\}\rangle$
- $\exists$-step plan: $\langle\{getKey, lightsOut\}, \{openDoor\}, \{goOut, closeDoor\}\rangle$
- relaxed $\exists$-step plan: $\langle\{getKey, lightsOut, openDoor, goOut\}, \{closeDoor\}\rangle$

2.2 ASP

In ASP, planning tasks are represented by facts. A solution of an ASP encoding is called *stable model*.

```
fluent(door; loc; light; key).
value(door, 0).  value(door, 1).
value(loc, in).  value(loc, out).
value(light, 0). value(light, 1).
value(key, 0).   value(key, 1).

init(door, 0). init(loc, in).  init(light, 1). init(key, 0).
goal(door, 0). goal(loc, out). goal(light, 0).

action(openDoor; closeDoor; goOut; lightOut; getKey).
prec(openDoor, door, 0).  prec(openDoor, key, 1).   post(openDoor, door, 1).
prec(closeDoor, door, 1).                           post(closeDoor, door, 0).
prec(goOut, loc, in).     prec(goOut, door, 1).  post(goOut, loc, out).
prec(lightOut, loc, in).  prec(lightOut, light, 1). post(lightOut, light, 0).
prec(getKey, loc, in).    prec(getKey, key, 0).     post(getKey, key, 1).
```

Listing 1: ASP encoding of the example planning task

Combining these facts with the right encodings, stable models correspond to the *sequential,* $\forall$*-step,* $\exists$*-step* or *relaxed* $\exists$*-step plans.*

```
#include <incmode>.
holds(X, V, 0) :- init(X, V).
#program check(t).
:- query(t), goal(X, V), not holds(X, V, t).
#program step(t).
{holds(X, V, t) : value(X, V)} = 1 :- fluent(X).
{occurs(A, t)} :- action(A).
:- occurs(A, t), post(A, X, V), not holds(X, V, t).
:- holds(X, V, t), not holds(X, V, t-1), not occurs(A, t) : post(A, X, V).
```

Listing 2: Core for sequential and parallel encodings for planning

All planning algorithms share a common incremental encoding using the incmode for clingo as seen in Listing 2 (Gebser et al. 2014). The program is divided into three subprograms: *base*, *step(t)* and *check(t)*.

The *base* subprogram contains the static part of the encoding, which is only grounded once. Line 2 defines the initial state s_0, where the predicate $holds(X, V, t)$ is used to represent fluents $X \in \mathcal{F}$ that have value $V \in dom(X)$ at timestep t.
check(t) is a parametrized subprogram starting at $t = 0$. The integrity constraint in Line 4 checks if $s_*(X) = s_t(X)\ \forall X \in dom(s_*)$. $Query(t)$ is true only for the latest integer t.
step(t) is grounded after check fails for each timestep t starting at $t = 1$. Note that postconditions of applied actions in timestep t are also set in t and the preconditions have to hold in $t-1$. Lines 6 defines holds for times step t, giving fluent X one value V from its domain and $occurs(A, t)$ in Line 7 are all possible actions in timestep t obeying rules 8 and 9. Line 8 ensures that the postcondition of an action is applied and also assert the confluence of any set of action applied in parallel since if $V = a^e(X) \neq \tilde{a}(X) = \tilde{V}$ either $holds(X, V, t)$ or $holds(X, \tilde{V}, t)$ would be false. Line 9 makes sure that fluents not affected by any action keep their value from the previous timestep. So far, the common core handles the transitions from one state to the next in regards of the postcondition of applied actions.

2.2.1 encoding for sequential plans

```
1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- #count{A : occurs(A, t)} > 1.
```

Listing 3: Extension for sequential plans

Starting with the extension for sequential plans as seen in Listing 3, Line 2 ensures that the precondition of an applied action is met in $t - 1$ and Line 3 only allows one action to occur per timestep.

Example 5. Running this with our example, we get three possible sequential plans.

```
Answer: 1
occurs(getKey,1) occurs(lightOut,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 3
occurs(lightOut,1) occurs(getKey,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
```

Listing 4: Output from clingo for sequential encoding

2.2.2 encoding for ∀-step plans

```
1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- occurs(A, t), prec(A, X, V), not post(A, X, _), not holds(X, V, t).
3 single(X, t) :- occurs(A, t), prec(A, X, V1), post(A, X, V2), V1 != V2.
4 :- single(X, t), #count{A : occurs(A, t), post(A, X, V)} > 1.
```

Listing 5: Extension for ∀-step plans

Now for the extension for ∀-step plans as seen in Listing 5, Line 1 is the same as in the sequential encoding. As mentioned before, the integrity constraint in Line 8 of the common core already ensures confluence of applied actions. Since the sequence of the actions has to be valid for all permutations, we have to make sure that no action invalidates the precondition of any other action. Let a be an action applied in parallel and $X \in dom(a^c)$. There are two cases to consider:

1. the fluent X remains unchanged, i.e. $X \in dom(a^c) \cap dom(a^e)$ and $a^c(X) = a^e(X)$ or $X \in dom(a^c)$ and $X \notin dom(a^e)$
2. the postcondition of a invalidates its precondition, i.e. $X \in dom(a^e) \cap a^c(X) \neq a^e(X)$ for an action a

Line 2 handles the first case, while the second case is a little trickier. If there is such an action, there can not be another action $\tilde{a}$ with $X \in dom(\tilde{a}^e)$, since if $a^e(X) = \tilde{a}^e(X)$, the precondition of a is invalidatet and if $a^e(X) \neq \tilde{a^e}(X)$, the confluence is undermined. For this, we define these actions described in (ii) as $single(X, t)$ in Line 3 and the integrity constraint in Line 4 ensures that there is at most one such action for each fluent X in timestep t.

Example 6. Running this with our example, we get two possible ∀-step plans.

```
Answer: 1
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,2) occurs(goOut,3) occurs(closeDoor,4)
Answer: 2
occurs(lightOut,1) occurs(getKey,1) occurs(openDoor,2) occurs(goOut,3) occurs(closeDoor,4)
```

Listing 6: Output from clingo for ∀-step encoding

2.2.3 encoding for ∃-step plans

```
1 :- occurs(A, t) , prec(A, X, V), not holds(X, V ,t -1).
2 apply(A1, t) :- action(A1),
3     ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.
4 ready(A, t) :- action(A), not occurs(A, t).
5 ready(A, t) :- apply(A, t).
6 :- action(A) , not ready(A, t).
```

Listing 7: Extension for ∃-step plans

```
1 :- occurs(A, t), prec(A ,X ,V), not holds(X, V, t-1).
2 #edge((A1, t), (A2, t)): occurs(A1, t),
3     post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.
```

Listing 8: Alternative Extension for ∃-step plans

There are two different ways to encode the extension for ∃-step plans as seen in Listing 7 and Listing 8. As before, Line 1 in Listing 7 and 8 is the same as in the sequential and ∀-step encoding. To ensure that there is at least one valid permutation, either an action a_1 that invalidates the precondition of another action a_2 has to be applied after a_2 or action a_2 does not occur in this timestep. For this, all actions has to be marked as *ready* (Line 6). An action is *ready* if it either does not occur in this timestep (Line 4) or if it is captured by *apply* (Line 2). An action can be safely applied, if all other actions whose preconditions it invalidates are ready, i.e. are either applied beforehand or not occurring in this timestep. Line 6 also ensures that actions does not circularly invalidates each other.

Example 7. Running this with our example, we get three possible ∃-step plans.

```
Answer: 1
occurs(getKey,1) occurs(openDoor,2) occurs(closeDoor,3) occurs(goOut,3) occurs(lightOut,3)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,2) occurs(closeDoor,3) occurs(goOut,3)
Answer: 3
occurs(lightOut,1) occurs(getKey,1) occurs(openDoor,2) occurs(closeDoor,3) occurs(goOut,3)
```

Listing 9: Output from clingo for ∃-step encoding

2.2.4 encoding for relaxed ∃-step plans

```
1 reach(X, V, t) :- holds (X, V, t -1).
2 reach(X, V, t) :- occurs(A, t), apply(A, t), post(A, X, V).
3 apply(A1, t) :- action(A1), reach(X, V, t) : prec(A1, X, V);
4     ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2 , V1 != V2.
5 ready(A, t) :- action(A), not occurs(A, t).
6 ready(A, t) :- apply(A, t).
7 :- action(A), not ready(A, t).
```

Listing 10: Extension for relaxed ∃-step plans

Finally, the extension for relaxed ∃-step plans as seen in Listing 10 is quite similar to the ∃-step encoding. The difference between those two is that the precondition of all actions do not have to hold simultaneous but they can "help" each other out. This is done by defining the predicate reach/3 in Line 1 and 2 as all fluents that can be reached by the step before or applied action in this timestep. An action can then be safely applied, if its precondition is reachable and as before, all actions that it invalidates are ready (Line 4). Again Line 7 ensures that applied action do not circularly invalidate each other.

Example 8. Running this with our example, we get three possible relaxed ∃-step plans.

```
Answer: 1
occurs(openDoor,1) occurs(getKey,1) occurs(closeDoor,2) occurs(goOut,2) occurs(lightOut,2)
Answer: 2
occurs(openDoor,1) occurs(lightOut,1) occurs(getKey,1) occurs(closeDoor,2) occurs(goOut,2)
Answer: 3
occurs(openDoor,1) occurs(goOut,1) occurs(lightOut,1) occurs(getKey,1) occurs(closeDoor,2)
```

Listing 11: Output from clingo for relaxed ∃-step encoding

2.3 Proof of Correctness

In order to prove the correctness of the presented encondings, we define the following:

- B is the rule in Line 1 of Listing 3.
- $Q(t)$ is the integrity constraint in the check program directive (Line 5) of Listing 2.
- $S(t)$ are the rules and integrity constraint in the step program directive (Lines 6-9) of Listing 2.
- $S^p(t)$ is the extension for the specific plans, where $p = s$ for Listing 3, $p = \forall$ for Listing 5, $p = \exists$ for Listing 7, and $p = R$ for Listing 10.
- I is the set of facts representing a planning task $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$.
- $\langle a_1, \ldots, a_n \rangle$ is a sequence of actions.
- $\langle A_1, \ldots, A_m \rangle$ is a a sequence of sets of actions.
- $P^p(n) := I \cup B \cup Q(0) \cup \bigcup_{t=1}^{n} (Q(t) \cup S(t) \cup S^p(t)) \cup \{query(n).\}$, where $p \in \{s, \forall, \exists, R\}$.

Theorem 1 (Correctness of sequential plans (DIMOPOULOS et al. 2019))**.**
$\langle a_1, \ldots, a_n \rangle$ is a sequential plan $\iff$ $P^s(n)$ has a stable model M, s.t.
$\{\langle a, t \rangle \mid occurs(a, t) \in M\} = \{\langle a_t, t \rangle \mid 1 \leq t \leq n\}$.

Proof. ($\Rightarrow$): Let $\langle a_1, \ldots, a_n \rangle$ be a sequential plan.
First we show that for any stable model M of $P^s(n)$ s.t. $\{\langle a, t \rangle | occurs(a, t) \in M\} = \{\langle a_t, t \rangle | 1 \leq t \leq n\}$, the rules and integrity constraints ensures that $\{\langle X, V, t \rangle \mid holds(X, V, t) \in M\} = \{\langle X, V, t \rangle \mid X \in \mathcal{F}, V = o(\langle a_1, \ldots, a_t \rangle, s_0), 0 \leq t \leq n\}$.

(i) the rule in B ensures that $holds(X, V, 0) = s_0(X)$ forall $X \in \mathcal{F}$

(ii) for any $1 \leq t \leq n$, both integrity constraints in S(t) ensure that $holds(X, V, t) = a_t^e(X)$ if $X \in dom(a_t^e)$ and $holds(X, V, t) = s_{t-1}(X)$ otherwise.

Therefore, the only stable model M that $P^s(n)$ can have is

$$M = I \cup \{occurs(a_t, t) \mid 1 \leq t \leq n\}$$
$$\cup \{holds(X, V, t) \mid X \in \mathcal{F}, v = o(\langle a_1, \ldots, a_t \rangle, s_0), 0 \leq t \leq n\} \cup \{query(n)\}.$$

s.t. $\{\langle a,t\rangle \mid occurs(a,t) \in M\} = \{\langle a_t,t\rangle \mid 1 \leq t \leq n\}$.
One can easily check that all rules and integrity constraints in $P^s(n)$ are satisfied and M is indeed a stable model of $P^s(n)$.
($\Leftarrow$): Let M be a stable model of $P^s(n)$ s.t. $\{\langle a,t\rangle \mid occurs(a,t) \in M\} = \{\langle a_t,t\rangle \mid 1 \leq t \leq n\}$.
We proof by induction over t, that for all $1 \leq t \leq n$:

$$\{\langle X,V\rangle \mid holds(X,V,t-1) \in M\} = \{\langle X,V\rangle \mid X \in \mathcal{F}, V = o(\langle a_1,\ldots,a_{t-1}\rangle, s_0)(X)\}.$$

I-Start: t=0
The Rule in B implies: $\{\langle X,V\rangle \mid holds(X,V,0) \in M\} = \{\langle X,V\rangle \mid X \in \mathcal{F}, s_0(X) = V\}$.
I-Step: $t \mapsto t+1$
The integrity constraint in Line 1 of Listing 3 in $S^s(t)$ ensures that the successor state of $s(t)$, i.e. $o(a_{t+1}, o(\langle a_1,\ldots,a_t\rangle, s_0))$ is defined. Additionally the choice rule in Line 6 of Listing 2 and the integrity constaints in $S(t)$ guarentee that

$$\{\langle X,V\rangle \mid holds(X,V,t+1) \in M\} = \{\langle X,V\rangle \mid X \in \mathcal{F}, V = o(\langle a_1,\ldots,a_{t+1}\rangle, s_0)(X)\}.$$

Finally, with the fact $query(n)$, the integrity constraint in $Q(n)$ yields that $o(\langle a_1,\ldots,a_n\rangle, s_0)(X) = s_*(X)$ for all $X \in dom(s_*)$. Thus, $\langle a_1,\ldots,a_n\rangle$ is a sequential plan. □

Theorem 2 (Correctness of parallel plans (DIMOPOULOS et al. 2019))**.**
$\langle A_1,\ldots,A_m\rangle$ is a $\forall$-/exists-/relaxed exists-step plan $\iff$ $P^p(m)$ has a stable model M s.t., $\{\langle a,t\rangle \mid occurs(a,t) \in M\} = \{\langle a,t\rangle \mid a \in A_t, 1 \leq t \leq m\}$ with $p \in \{\forall, \exists, R\}$.

Proof. ($\Rightarrow$) $\langle A_1,\ldots,A_m\rangle$ is a $\forall$-/*exists*-/*relaxed exists*-step plan.
For each $1 \leq t \leq m$, let s_t be the state s.t. $s_t(X) = \begin{cases} a^e(X) & \forall a \in A_t \text{ and } X \in dom(a^e) \\ s_{t-1}(X) & \forall X \in \mathcal{F} \setminus \bigcup_{a \in A_t} dom(a^e) \end{cases}$.
Then, as in the proof above, for any stable model M of $P^p(m)$ s.t. $\{\langle a,t\rangle \mid occurs(a,t) \in M\} = \{\langle a,t\rangle \mid a \in A_t, 1 \leq t \leq m\}$, the rule in B together with the choice rule and the integrity constraints in $S(t)$ make sure that $\{\langle X,V,t\rangle \mid holds(X,V,t) \in M\} = \{\langle X,V,t\rangle \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\}$. We now split the proof into the different cases.

Case 1: $\langle A_1,\ldots,A_m\rangle$ is a $\forall-$step plan.
The atom $single(X,t)$ is derived by the rule in $S^\forall(t) \iff \exists a \in A_i$ s.t. $X \in dom(a^c) \cap dom(a^e)$ and $a^c(X) \neq a^e(X)$ for $1 \leq t \leq m$. In other words, if there is an action that invalidates its own

precondition. Hence $P^{\forall}(m)$ cannot have any stable model M apart from

$$\begin{aligned} M = I \cup \{&occurs(a,t) \mid a \in A_t, 1 \leq t \leq m\} \\ &\cup \{holds(X,V,t) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \\ &\cup \{single(X,t) \mid X \in dom(a^c) \cap dom(a^e), a^c(X) \neq a^e(X), a \in A_t, 1 \leq t \leq m\} \\ &\cup \{query(m).\} \end{aligned}$$

s.t. $\{\langle a,t\rangle \mid occurs(a,t) \in M\} = \{\langle a,t\rangle \mid a \in A_t, 1 \leq t \leq m\}$. Since for any $1 \leq t \leq m$, the successor state $s_t = o(\langle a_1,\ldots,a_k\rangle, s_{t-1})$ is defined for any sequence $\langle a_1,\ldots,a_k\rangle$ s.t. $\{a_1,\ldots,a_k\} = A_t$, we have that M satisfies all integrity constraints in $S^{\forall}(t)$. As above, one can easily check that all other rules and integrity constraints in $P^{\forall}(m)$ are satisfied as well. Therefore M is a stable model of $P^{\forall}(m)$.

Case 2: $\langle A_1,\ldots,A_m\rangle$ is a $\exists$-step plan.
Then for any $1 \leq t \leq m, \exists$ a sequence of actions $\langle a_1,\ldots,a_k\rangle$, s.t. $\{a_1,\ldots,a_k\} = A_t$ and $a_i^e(X) = \tilde{a}_j^c(X)$ if $X \in dom(a_i^e) \cap dom(\tilde{a}_j^c)$ for any $1 \leq i < j \leq m$. That is the edges of the directed graph:

$$G_t = (\mathcal{O}, \{\langle a, \tilde{a}\rangle \mid a \in A_t, \tilde{a} \in \mathcal{O} \setminus \{a\}, X \in dom(a^e) \cap dom(\tilde{a}^c), a^e(X) \neq \tilde{a}^c(X)\})$$

can connect some a_i for $1 \leq i \leq k$ to elements of $\mathcal{O} \setminus \{a_i,\ldots,a_k\}$ only if G_t remains acyclic. The acyclicity of G_t yields that the recursive derivation of $apply(a,t)$ and $ready(a,t)$ is successful, i.e. they both belong to the least fixpoint in $S^{\exists}(t)$ (Line 2-5 in 7) for each $a \in mathcalO$ and $1 \leq t \leq m$ and thus, $P^{\exists}(m)$ cannot have any stable model apart from

$$M' = M \cup \{apply(a,t) \mid a \in \mathcal{O}, 1 \leq t \leq m\} \cup \{ready(a,t) \mid a \in \mathcal{O}, 1 \leq t \leq m\}$$

s.t. $\{\langle a,t\rangle \mid occurs(a,t) \in M'\} = \{\langle a,t\rangle \mid a \in A_t, 1 \leq t \leq m\}$. As above, one can easily check that all other rules and integrity constraints in $P^{\exists}(m)$ are satisfied as well. Therefore M is a stable model of $P^{\exists}(m)$.

Case 3: $\langle A_1,\ldots A_m\rangle$ is a *relaxed* $\exists$-step plan. Then for any $1 \leq t \leq m, \exists$ sequence of actions $\langle a_1,\ldots,a_k\rangle$ s.t. $\{a_1,\ldots,a_k\} = A_t$ and $o(a_i, \langle a_1,\ldots,a_{i-1}\rangle, s_{t-1})$ is defined for $1 \leq i \leq k$, which implies that $a_i^e(X) = a_j^c(X)$ if $X \in dom(a_i^e) \cap dom(a_j^c)$ for $i < j \leq k$. Given this, the least fixpoint of $S^R(t)$ (Lines 1-6 in Listing 10) contains:

(i) $reach(X,V,t)$ for each $X \in \mathcal{F}$ and $V \in \{s_{t-1}(X), s_t(X)\}$
(ii) $apply(a,t)$ for each $a \in \mathcal{O}$ s.t. $a^c(X) \in \{s_{t-1}, s_t\}$ $\forall X \in dom(a^c)$
(iii) $ready(a,t)$ for each $a \in \mathcal{O}$

Thus, the only stable model $P^R(m)$ can have is

$$\begin{aligned} M = I &\cup \{occurs(a,t) \mid a \in A_t, 1 \leq t \leq m\} \\ &\cup \{holds(X,V,t) \mid X \in \mathcal{F}, s_t(X) = V, 0 \leq t \leq m\} \\ &\cup \{reach(X,V,t) \mid X \in \mathcal{F}, V \in \{s_{t-1}, s_t\}, 1 \leq t \leq m\} \\ &\cup \{apply(a,t) \mid a \in \mathcal{O} : a^c(X) \in \{s_{t-1}(X), s_t(X)\}\ \forall X \in dom(a^c)\} \\ &\cup \{ready(a,t) \mid a \in \mathcal{O}, 1 \leq t \leq m\} \\ &\cup \{query(m)\} \end{aligned}$$

s.t. $\{\langle a,t\rangle \mid occurs(a,t) \in M'\} = \{\langle a,t\rangle \mid a \in A_t, 1 \leq t \leq m\}$. As above, one can easily check that all other rules and integrity constraints in $P^R(m)$ are satisfied as well. Therefore M is a stable model of $P^R(m)$.
($\Leftarrow$) M is a stable model of $P^p(m)$ s.t. $\{\langle a,t\rangle \mid occurs(a,t) \in M'\} = \{\langle a,t\rangle \mid a \in A_t, 1 \leq t \leq m\}$. For any $1 \leq t \leq m$, the choice rule in Line 6 of Listing 2 makes sure that s_t s.t. $s_t(X) = V \iff holds(X,V,t) \in M$ is a state.

□

3 Method

3.1 Pipeline Overview

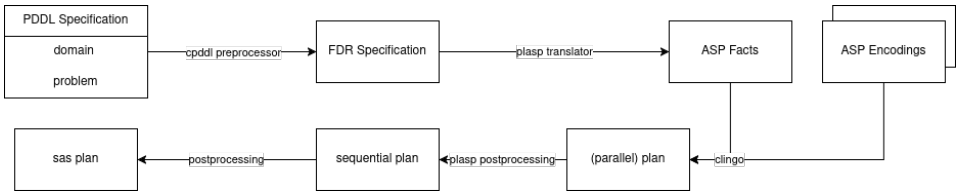

Figure 3.1: Pipeline for testing the different planning encodings

To test and compare t he different parallel planning encodings against each other and against the sequential planning encoding, we set up a python pipeline using the lab framework (Seipp et al. 2017) and the unofficial collection of (sequential) IPC benchmark instances from the downward project [1].
Starting at a planning task in the Planning Domain Definition Language (PDDL), we first preprocess it using cpddl (Fišer 2025) that converts it into a finite domain representation (FDR). Plasp then takes the FDR as input and translates it into ASP facts representing the planning task. Further information of this step can be found in (DIMOPOULOS et al. 2019). Using clingo with a modified version of the incremental encodings presented above together with the facts from the output of plasp, we can then compute sequential, $\forall$-step, $\exists$-step or relaxed $\exists$-step plans as stable models. Finally, to validate the given plan, we postprocess the parallel representation back into a sequential plan, using the postprocess encoding provided by plasp and validate the plan with VAL [2]. Each step has some optimizations and configurations that will be explained in the following sections.

3.2 Optimizations

3.2.1 Mutexes

Definition 7 ***mutex and mutex groups.*** A set of facts $M \subseteq \mathcal{F}$ is called

(i) a *mutex* $:\Longleftrightarrow$ for all reachable states $s : M \not\subseteq s$.

[1] https://github.com/aibasel/downward-benchmarks
[2] https://github.com/KCL-Planning/VAL

(ii) a *mutex group* $:\Longleftrightarrow$ for all reachable states $s : |M \cap s| \leq 1$.

In other words, a mutex is a set of fluents that can never be true at the same time in any reachable state, while a mutex group is a set of fluents where at most one fluent can be true at the same time. Mutexes and mutex groups can be used to optimize the planning encodings by reducing the search space. For this, we extend our endcoding in Listing 2 with the following rules:

```
1  #program base.
2  ...
3  mutex(G,X) :- mutexGroup(G), contains(G,X,V), fluent(X,V).
4  mutex(G)   :- mutexGroup(G), #count{X : mutex(G,X)} > 1.
5  :- mutex(G), #count{X,V : holds(X,V,0), contains(G,X,V)} > 1.^
6  ...
7  #program step(t).
8  ...
9  :- mutex(G), #count{X,V : holds(X,V,t), contains(G,X,V)} > 1.
10 ...
```

Listing 12: Expansion for mutexes

Here, the mutex group G is given by plasp as facts of the form *mutexGroup*/1 and *contains*/3. In the base program, we first define which fluents belong to a mutex group (Line 3), where a group must contain at least two fluents (Line 4). The integrity constraint in Line 5 ensures that no more than one fluent of each mutex group holds in the initial state and the integrity constraint in Line 9 ensures the same for each state in timestep t.

3.2.2 Guess and Check

An optimization specific to the (relaxed) $\exists$-step encodings is the guess and check approach. The general idea is to first guess a solution without looking for actions that circularly invalidate each other and afterwards check if the guessed solution has a cycle. Formally, we encode the problem by a pair of programs $\langle G, C \rangle$ where G is the guessing program and C is the checking program. Stable models M of G yields a solution if $C \cap M$ is $unsatisfiable$. For making "systematic" guesses, (DIMOPOULOS et al. 2019) proposes to use the core program ($S(t)$) in Listing 2 together with the integrity constraint in Line 1 of Listing 3, 5 or 7 (B) and the facts representing a planning task. Thus, a stable model M of G is s.t. the pre- and postconditons hold for all actions applied in parallel (Line 8 of Listing 2 and Line 1 of Listing 3, 5 or 7). and the integrity constraint in Line 8 of Listing 2 also ensures confluence of applied actions. The only thing

remained to be checked is if actions circularly invalidate each other, i.e. $a^e(X) \neq \tilde{a}^c(X)$ and $\tilde{a}^e(\tilde{X}) \neq \tilde{a}^c(\tilde{X})$. This can be accomplished by taking the atoms over $occurs/2$ from M as facts together with a program C containing the facts representing a planning task and the following encoding:

```
1  #include <incmode>.
2  #program check(t).
3  :- query(t), not cycle(t).
4  #program step(t).
5  prec(A, X, V, t) :- prec(A, X, V), occurs(A, t).
6  post(A, X, V, t) :- post(A, X, V), occurs(A, t).
7  action(A, t) :- occurs(A, t).
8  apply(A1, t) :- action(A1, t),
9      ready(A2, t): post(A1, X, V1, t), prec(A2, X, V2, t), A1 != A2 , V1 != V2.
10 ready(A, t) :- action(A, t), not occurs (A, t).
11 ready (A, t) :- apply(A, t).
12 cycle(t) :- action (A, t), not ready (A, t).
```

Listing 13: Encoding for checking circular invalidation

Here, the maximum value for t in subprogram *check* and *step* is $m = max\{t \mid occurs(a,t)\}$ and C becomes $unsatisfiable$ if $t > m$. Like in Listing 7, we check for each t if all actions $A \in occurs(A,t)$ can be applied by the predicate *ready* (Line 7-11). If actions in A circularly invalidate each other, $ready(A,t)$ is underivable for these actions and thus $cycle(t)$ (Line 12) in step t is derived. Hence, C becomes satisfiable if there is a circular invalidation in step t, which means that there is no permutation of the actions A, s.t. the successor state is defined and therefore not (relaxed) $\exists$-step serializable.

3.2.3 Heuristic

The general idea of heuristics in clingo described by (DIMOPOULOS et al. 2019) is to extend the search heuristic by giving each atom a

- *level* ($\in \mathbb{Z}$, 0 by default), allowing to modify the order of which atoms are selected first
- *sign* ($\in \mathbb{Z}$, 0 by default, where values > 0 corresponds to $true$, < 0 to $false$ and 0 to $undefined$), which allows for controlling the truth value assigned to atoms.

During solving, these values get modified during the activation of #**heuristic** directives. The search heuristic of clingo selects the atom with the highest level and sets it to its associated *sign*, if *sign* defined, else switching to the default *sign* heuristic.

```
1 #program step(t).
2 #heuristic holds(X, V, t-1):    holds(X, V, t). [2147483647-t, true]
3 #heuristic holds(X, V, t-1):    not holds(X, V, t). [2147483647-t, false]
```

Listing 14: backward progpagating heuristic

The heuristic in Listing 14 aims starting at the goal of a planning task and then searching backwards. Here both lines have the level $l = 2^{31} - 1 - t$ where $2^{31} - 1$ is the largest integer supported by clingo, giving the starting steps the highest priority. The second line expresses that whenever clingo made $holds(X, V, t)$ true, $holds(X, V, t-1)$ should be made true at the level l as well. Similarly, the directive in Line 3 is actived when an instance of $holds(X, V, t)$ becomes false, in which case $holds(X, V, t-1)$ should be made false at level l as well. Both directives associates atoms over holds/3 with higher level the earlier the state is, i.e. the value for t is smaller, which intends to bias the search of *clingo* to establish the goal conditions as early as possible.

3.3 Experiment

Setup + Configs

4 Results

4.1 One to One Comparison

Bibliography

Helmert, M. (2006). "The Fast Downward Planning System". In: *Journal of Artifical Intelligence Research* 26, pp. 191–246.

Gebser, Martin et al. (May 2014). *Clingo = ASP + control: Preliminary report*. Tech. rep. 1405.3694. arXiv. URL: https://arxiv.org/abs/1405.3694v1.

Seipp, Jendrik et al. (2017). *Downward Lab*. https://doi.org/10.5281/zenodo.790461.

DIMOPOULOS, YANNIS et al. (Jan. 2019). "plasp 3: Towards Effective ASP Planning". In: *Theory and Practice of Logic Programming* 19.3, pp. 477–504. ISSN: 1475-3081. DOI: 10.1017/s1471068418000583. URL: http://dx.doi.org/10.1017/S1471068418000583.

Fišer, Daniel (Nov. 2025). *cpddl*. Version 1.5. DOI: 10.5281/zenodo.16423302. URL: https://doi.org/10.5281/zenodo.16423302.